

в Министерстве науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная
техника»

УТВЕРЖДАЮ
Зав. кафедрой ИТАС
докт. экон. наук, профессор
_____ (Р.А.Файзрахманов)
«___» _____ 2026 г.

**Информатика в приложении к отрасли
Лабораторная работа №1**

Выполнил студент гр. ИВТ-25-16
_____ **Тагиров Вадим Эльвирович** _____
(Фамилия, Имя, Отчество)

(подпись)

Проверил:

_____ **Проф. Каф. ИТАС** _____
_____ **Шварц К. Г** _____
(должность, Ф.И.О руководителя от кафедры)

_____ (оценка) _____ (подпись)

(дата)

г. Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Теоретическая часть.....	4
1.1 Общие понятия.....	4
1.2 Метод Гаусса.....	4
1.2.1 Идея метода.....	4
1.2.2 Прямой и обратный ход.....	4
1.2.3 Выбор главного элемента.....	4
1.2.4 Нормирование.....	4
1.3 Метод Зейделя.....	5
1.3 Способы задания множеств.....	5
2 Постановка задачи.....	6
3 Краткое описание метода.....	7
3.1 Метод Гаусса.....	7
3.2 Метод Зейделя.....	7
5 Результаты вычислений с пошаговым представлением.....	9
5.1 Метод Гаусса с выбором главного элемента и нормированием.....	9
5.2 Метод Зейделя.....	10
6 Обсуждение результатов.....	14
Приложение.....	16
ПРИЛОЖЕНИЕ А.....	24

ВВЕДЕНИЕ

Системы линейных алгебраических уравнений (СЛАУ) занимают одно из центральных мест в вычислительной математике. К ним сводится решение множества прикладных задач: расчёт электрических цепей, задач теплопроводности и механики, балансовых моделей в экономике, обработки экспериментальных данных. В связи с этим разработка и анализ методов решения СЛАУ является важной задачей.

Методы решения СЛАУ делятся на две большие группы: *прямые* (точные) методы, позволяющие получить решение за конечное число шагов, и *итерационные* методы, дающие решение с заданной точностью в результате сходящегося вычислительного процесса.

Цель работы: освоить методы решения систем линейных алгебраических уравнений и применить их на практике.

Задачи работы:

- 1) Изучить метод Гаусса с выбором главного элемента и нормированием.
- 2) Изучить метод Зейделя.
- 3) Решить заданную систему варианта №26 обоими методами с пошаговым представлением вычислений.
- 4) Сравнить результаты, проверить их подстановкой и проанализировать точность.

1 Теоретическая часть

1.1 Общие понятия

Условие существования и единственности решения: система имеет единственное решение тогда и только тогда, когда $\det A \neq 0$ (матрица невырождена).

1.2 Метод Гаусса

1.2.1 Идея метода

Метод Гаусса (метод последовательного исключения неизвестных) основан на приведении матрицы системы к треугольному или диагональному виду с помощью элементарных преобразований строк:

- 1) Перестановка строк
- 2) Умножение строки на ненулевое число
- 3) прибавление к одной строке другой, умноженной на число

Эти преобразования *не меняют решения* системы.

1.2.2 Прямой и обратный ход

- 1) Прямой ход — система приводится к верхней треугольной матрице:
- 2) Обратный ход — последовательно находятся неизвестные

1.2.3 Выбор главного элемента

Если на очередном шаге диагональный элемент a_{kk} мал или равен нулю, возникает проблема: деление на малое число приводит к большим погрешностям, а деление на ноль невозможно.

Решение: на каждом шаге выбирается **главный элемент** — наибольший по модулю элемент в текущем столбце (начиная с k -й строки). Строка с главным элементом переставляется на позицию k .

1.2.4 Нормирование

После выбора главного элемента строка k **нормируется** — делится на главный элемент a_{kk} , чтобы на диагонали получилась 1. Затем столбец k обнуляется **во всех** остальных строках (не только ниже диагонали). В результате после всех шагов слева получается **единичная матрица**, а справа — решение.

1.3 Метод Зейделя

Метод Зейделя — модификация метода простой итерации. Основное отличие: при вычислении $(k+1)$ -го приближения x_i используются уже найденные $(k+1)$ -е приближения $x_1, \dots, x_{i-1}$.

Достаточное условие сходимости метода Зейделя: $\|C\| < 1$ хотя бы в одной матричной форме. Если матрица A обладает *диагональным преобладанием*, метод Зейделя сходится при любом начальном приближении.

Итерации прекращаются, когда достигнута заданная точность ε .

Метод Зейделя обычно **сходится быстрее** метода простой итерации, так как использует более свежие значения неизвестных. Однако в некоторых случаях метод простой итерации может оказаться устойчивее.

Общее число итераций зависит от матрицы C и требуемой точности. Для хорошо обусловленных систем с диагональным преобладанием обычно требуется 10–50 итераций.

1.3 Способы задания множеств

В работе реализованы три способа задания множеств:

- 1) ручной ввод — пользователь последовательно вводит элементы;
- 2) случайная генерация — программа случайным образом выбирает элементы из универсума;
- 3) задание по условиям — программа фильтрует универсум по заданным пользователем условиям (положительные/отрицательные, чётные/нечётные, кратные N , диапазон).

2 Постановка задачи

Дано: система линейных алгебраических уравнений с тремя неизвестными:

$$\begin{cases} 0.9x_1 + 2.7x_2 - 3.8x_3 = 2.4 \\ 2.5x_1 + 5.8x_2 - 0.5x_3 = 3.5 \\ 4.5x_1 - 2.1x_2 + 3.2x_3 = -1.2 \end{cases}$$

В матричном виде $Ax = b$:

$$A = \begin{pmatrix} 0.9 & 2.7 & -3.8 \\ 2.5 & 5.8 & -0.5 \\ 4.5 & -2.1 & 3.2 \end{pmatrix}, \quad b = \begin{pmatrix} 2.4 \\ 3.5 \\ -1.2 \end{pmatrix}.$$

Найти: вектор неизвестных $x=(x_1, x_2, x_3)^T$.

Анализ задачи:

- Число уравнений равно числу неизвестных ($n=3$), матрица квадратная.
- Определитель матрицы A отличен от нуля, следовательно, решение существует и единственно.
- Требуется решить систему двумя методами:
 1. Прямым — методом Гаусса с выбором главного элемента и нормированием.
 2. Итерационным — методом Зейделя с точностью $\varepsilon=0.0001$.

3 Краткое описание метода

3.1 Метод Гаусса

Метод Гаусса — прямой метод решения СЛАУ, основанный на последовательном исключении неизвестных. Система приводится к диагональному виду, после чего решение находится без обратного хода.

Алгоритм:

1. Строится расширенная матрица системы $[A|b]$.
2. На каждом шаге k ($k=1, \dots, n$) среди элементов текущего столбца (начиная с k -й строки) выбирается **наибольший по модулю** элемент — *главный элемент*.
3. Строка с главным элементом переставляется на позицию k .
4. Строка k **нормируется**: все её элементы делятся на главный элемент, чтобы на диагонали получилась 1.
5. Столбец k **обнуляется** во всех остальных строках: для каждой строки $i \neq k$ вычисляется множитель $m_i = a_{ik}$ и из строки i вычитается m_i -кратная строка k .
6. После выполнения всех шагов слева получается единичная матрица, а в последнем столбце — решение.

Условие применимости: матрица A невырождена ($\det A \neq 0$). Выбор главного элемента повышает устойчивость к ошибкам округления.

3.2 Метод Зейделя

Метод Зейделя — итерационный метод. Система $Ax=b$ преобразуется к виду « $x = Cx + d$ » где C — матрица коэффициентов, d — вектор свободных членов.

Особенность: при вычислении $(k+1)$ -го приближения x_i используются **уже найденные** $(k+1)$ -е приближения $x_1, \dots, x_{i-1}$.

Условие сходимости: достаточно, чтобы выполнялось $\|C\| < 1$ хотя бы в одной матричной норме

Приведение системы к виду с диагональным преобладанием.
Для сходимости метода Зейделя исходная система преобразуется линейной комбинацией уравнений так, чтобы диагональные коэффициенты преобладали по модулю над суммой остальных коэффициентов строки/

4 Исходные данные для конкретных расчётов

Исходная система (вариант №26):

$$\begin{cases} 0.9x_1 + 2.7x_2 - 3.8x_3 = 2.4 \\ 2.5x_1 + 5.8x_2 - 0.5x_3 = 3.5 \\ 4.5x_1 - 2.1x_2 + 3.2x_3 = -1.2 \end{cases}$$

Преобразование системы для метода Зейделя.
Исходная матрица не обладает диагональным преобладанием. Выполним линейные комбинации уравнений:

(2) + (3):

$$7.0x_1 + 3.7x_2 + 2.7x_3 = 2.3$$

$$\text{Проверка: } |7.0| > |3.7| + |2.7| = 6.4 \checkmark$$

(1) + (2):

$$3.4x_1 + 8.5x_2 - 4.3x_3 = 5.9$$

$$\text{Проверка: } |8.5| > |3.4| + |4.3| = 7.7 \checkmark$$

(1):

$$0.9x_1 + 2.7x_2 - 3.8x_3 = 2.4$$

$$\text{Проверка: } |-3.8| > |0.9| + |2.7| = 3.6 \checkmark$$

Полученная эквивалентная система:

$$\begin{cases} 7.0x_1 + 3.7x_2 + 2.7x_3 = 2.3 \\ 3.4x_1 + 8.5x_2 - 4.3x_3 = 5.9 \\ 0.9x_1 + 2.7x_2 - 3.8x_3 = 2.4 \end{cases}$$

Эта система имеет диагональное преобладание и пригодна для метода Зейделя.

5 Результаты вычислений с пошаговым представлением

5.1 Метод Гаусса с выбором главного элемента и нормированием

=== МЕТОД ГАУССА С ВЫБОРОМ ГЛАВНОГО ЭЛЕМЕНТА И НОРМИРОВАНИЕМ ===

Исходная расширенная матрица:

	0.9000	2.7000	-3.8000	2.4000	
	2.5000	5.8000	-0.5000	3.5000	
	4.5000	-2.1000	3.2000	-1.2000	

--- Шаг 1 ---

Ищем главный элемент в столбце 1.

Максимальный по модулю: $a[3][1] = 4.5000$

Переставляем строки 1 и 3:

	4.5000	-2.1000	3.2000	-1.2000	
	2.5000	5.8000	-0.5000	3.5000	
	0.9000	2.7000	-3.8000	2.4000	

Нормируем строку 1: делим все элементы на 4.5000 ($a[1][1]$)

	1.0000	-0.4667	0.7111	-0.2667	
	2.5000	5.8000	-0.5000	3.5000	
	0.9000	2.7000	-3.8000	2.4000	

Обнуляем столбец 1 в остальных строках.

Для строки 2: множитель = 2.5000, вычитаем 2.5000 * строку 1

Для строки 3: множитель = 0.9000, вычитаем 0.9000 * строку 1

	1.0000	-0.4667	0.7111	-0.2667	
	0.0000	6.9667	-2.2778	4.1667	
	0.0000	3.1200	-4.4400	2.6400	

--- Шаг 2 ---

Ищем главный элемент в столбце 2.

Максимальный по модулю: $a[2][2] = 6.9667$

Перестановка не требуется.

Нормируем строку 2: делим все элементы на 6.9667 ($a[2][2]$)

	1.0000	-0.4667	0.7111	-0.2667	
	0.0000	1.0000	-0.3270	0.5981	
	0.0000	3.1200	-4.4400	2.6400	

Обнуляем столбец 2 в остальных строках.

Для строки 1: множитель = -0.4667, вычитаем $-0.4667 * \text{строку } 2$

Для строки 3: множитель = 3.1200, вычитаем $3.1200 * \text{строку } 2$

	1.0000	0.0000	0.5585	0.0124	
	0.0000	1.0000	-0.3270	0.5981	
	0.0000	0.0000	-3.4199	0.7740	

--- Шаг 3 ---

Ищем главный элемент в столбце 3.

Максимальный по модулю: $a[3][3] = -3.4199$

Перестановка не требуется.

Нормируем строку 3: делим все элементы на -3.4199 ($a[3][3]$)

	1.0000	0.0000	0.5585	0.0124	
	0.0000	1.0000	-0.3270	0.5981	
	-0.0000	-0.0000	1.0000	-0.2263	

Обнуляем столбец 3 в остальных строках.

Для строки 1: множитель = 0.5585, вычитаем $0.5585 * \text{строку } 3$

Для строки 2: множитель = -0.3270, вычитаем $-0.3270 * \text{строку } 3$

	1.0000	0.0000	0.0000	0.1388	
	0.0000	1.0000	0.0000	0.5241	
	-0.0000	-0.0000	1.0000	-0.2263	

Решение (Гаусс): 0.1388 0.5241 -0.2263

5.2 Метод Зейделя

=== МЕТОД ЗЕЙДЕЛЯ ===

Система, приведённая к виду с диагональным преобладанием:

	7.0000	3.7000	2.7000	2.3000	
	3.4000	8.5000	-4.3000	5.9000	
	0.9000	2.7000	-3.8000	2.4000	

Проверка условия сходимости $||C||_{\infty} < 1$:

Строка 1: $|a_{12}/a_{11}| + |a_{13}/a_{11}| = 0.9143$

Строка 2: $|a_{21}/a_{22}| + |a_{23}/a_{22}| = 0.9059$

Строка 3: $|a_{31}/a_{33}| + |a_{32}/a_{33}| + = 0.9474$

Норма $||C||_{\infty} = 0.9474 < 1$ – условие сходимости выполнено.

Приведение к виду $x = Cx + d$:

$x_1 = -0.5286x_2 - 0.3857x_3 + 0.3286$

$x_2 = -0.4000x_1 + 0.5059x_3 + 0.6941$

$x_3 = 0.2368x_1 + 0.7105x_2 - 0.6316$

Начальное приближение: $x = (0, 0, 0)$

Итерации:

к	x1	x2	x3	погрешность
---	----	----	----	-------------

Итерация 1:

$x_1 = 2.3000/7.0000 - 0.5286*0.0000 - 0.3857*0.0000 = 0.3286$

$x_2 = 5.9000/8.5000 - 0.4000*0.3286 + 0.5059*0.0000 = 0.5627$

$x_3 = 2.4000/-3.8000 + 0.2368*0.3286 + 0.7105*0.5627 = -0.1540$

Погрешность: $|x_1 - x_{1_old}| + |x_2 - x_{2_old}| + |x_3 - x_{3_old}| = 1.0452$

1	0.328571	0.562689	-0.153954	1.045215
---	----------	----------	-----------	----------

Итерация 2:

$x_1 = 2.300000/7.000000 - 0.528571*0.562689 - 0.385714*-0.153954 = 0.090532$

$x_2 = 5.900000/8.500000 - 0.400000*0.090532 + 0.505882*-0.153954 = 0.580022$

$x_3 = 2.400000/-3.800000 + 0.236842*0.090532 + 0.710526*0.580022 = -0.198016$

Погрешность: $|x_1 - x_{1_old}| + |x_2 - x_{2_old}| + |x_3 - x_{3_old}| = 0.299434$

2	0.090532	0.580022	-0.198016	0.299434
---	----------	----------	-----------	----------

Итерация 3:

$x1 = 2.300000/7.000000 - 0.528571*0.580022 - 0.385714*-0.198016 = 0.098366$
 $x2 = 5.900000/8.500000 - 0.400000*0.098366 + 0.505882*-0.198016 = 0.554598$
 $x3 = 2.400000/-3.800000 + 0.236842*0.098366 + 0.710526*0.554598 = -0.214225$
Погрешность: $|x1 - x1_{old}| + |x2 - x2_{old}| + |x3 - x3_{old}| = 0.049466$
3 0.098366 0.554598 -0.214225 0.049466

Итерация 4:

$x1 = 2.300000/7.000000 - 0.528571*0.554598 - 0.385714*-0.214225 = 0.118056$
 $x2 = 5.900000/8.500000 - 0.400000*0.118056 + 0.505882*-0.214225 = 0.538523$
 $x3 = 2.400000/-3.800000 + 0.236842*0.118056 + 0.710526*0.538523 = -0.220984$
Погрешность: $|x1 - x1_{old}| + |x2 - x2_{old}| + |x3 - x3_{old}| = 0.042525$
4 0.118056 0.538523 -0.220984 0.042525

Итерация 5:

$x1 = 2.300000/7.000000 - 0.528571*0.538523 - 0.385714*-0.220984 = 0.129160$
 $x2 = 5.900000/8.500000 - 0.400000*0.129160 + 0.505882*-0.220984 = 0.530662$
 $x3 = 2.400000/-3.800000 + 0.236842*0.129160 + 0.710526*0.530662 = -0.223939$
Погрешность: $|x1 - x1_{old}| + |x2 - x2_{old}| + |x3 - x3_{old}| = 0.021921$
5 0.129160 0.530662 -0.223939 0.021921

Итерация 6:

$x1 = 2.300000/7.000000 - 0.528571*0.530662 - 0.385714*-0.223939 = 0.134455$
 $x2 = 5.900000/8.500000 - 0.400000*0.134455 + 0.505882*-0.223939 = 0.527049$
 $x3 = 2.400000/-3.800000 + 0.236842*0.134455 + 0.710526*0.527049 = -0.225252$
Погрешность: $|x1 - x1_{old}| + |x2 - x2_{old}| + |x3 - x3_{old}| = 0.010221$
6 0.134455 0.527049 -0.225252 0.010221

Итерация 7:

$x1 = 2.300000/7.000000 - 0.528571*0.527049 - 0.385714*-0.225252 = 0.136872$
 $x2 = 5.900000/8.500000 - 0.400000*0.136872 + 0.505882*-0.225252 = 0.525418$
 $x3 = 2.400000/-3.800000 + 0.236842*0.136872 + 0.710526*0.525418 = -0.225839$
Погрешность: $|x1 - x1_{old}| + |x2 - x2_{old}| + |x3 - x3_{old}| = 0.004633$
7 0.136872 0.525418 -0.225839 0.004633

Итерация 8:

```
x1 = 2.300000/7.000000 - 0.528571*0.525418 - 0.385714*-0.225839 = 0.137960
x2 = 5.900000/8.500000 - 0.400000*0.137960 + 0.505882*-0.225839 = 0.524686
x3 = 2.400000/-3.800000 + 0.236842*0.137960 + 0.710526*0.524686 = -0.226101
Погрешность: |x1 - x1_old| + |x2 - x2_old| + |x3 - x3_old| = 0.002082
8      0.137960      0.524686      -0.226101      0.002082
```

Итерация 9:

```
x1 = 2.300000/7.000000 - 0.528571*0.524686 - 0.385714*-0.226101 = 0.138448
x2 = 5.900000/8.500000 - 0.400000*0.138448 + 0.505882*-0.226101 = 0.524358
x3 = 2.400000/-3.800000 + 0.236842*0.138448 + 0.710526*0.524358 = -0.226219
Погрешность: |x1 - x1_old| + |x2 - x2_old| + |x3 - x3_old| = 0.000933
9      0.138448      0.524358      -0.226219      0.000933
```

Итерация 10:

```
x1 = 2.300000/7.000000 - 0.528571*0.524358 - 0.385714*-0.226219 = 0.138667
x2 = 5.900000/8.500000 - 0.400000*0.138667 + 0.505882*-0.226219 = 0.524211
x3 = 2.400000/-3.800000 + 0.236842*0.138667 + 0.710526*0.524211 = -0.226271
Погрешность: |x1 - x1_old| + |x2 - x2_old| + |x3 - x3_old| = 0.000418
10     0.138667      0.524211      -0.226271      0.000418
```

Итерация 11:

```
x1 = 2.300000/7.000000 - 0.528571*0.524211 - 0.385714*-0.226271 = 0.138764
x2 = 5.900000/8.500000 - 0.400000*0.138764 + 0.505882*-0.226271 = 0.524145
x3 = 2.400000/-3.800000 + 0.236842*0.138764 + 0.710526*0.524145 = -0.226295
Погрешность: |x1 - x1_old| + |x2 - x2_old| + |x3 - x3_old| = 0.000187
11     0.138764      0.524145      -0.226295      0.000187
```

Итерация 12:

```
x1 = 2.300000/7.000000 - 0.528571*0.524145 - 0.385714*-0.226295 = 0.138808
x2 = 5.900000/8.500000 - 0.400000*0.138808 + 0.505882*-0.226295 = 0.524116
x3 = 2.400000/-3.800000 + 0.236842*0.138808 + 0.710526*0.524116 = -0.226305
Погрешность: |x1 - x1_old| + |x2 - x2_old| + |x3 - x3_old| = 0.000084
12     0.138808      0.524116      -0.226305      0.000084
```

Погрешность 0.000084 <= 0.000100 – итерации завершены.

Решение (Зейдель): 0.138808 0.524116 -0.226305

6 Обсуждение результатов

Сравнение:

Метод	x1	X2	X3
Гаусса	0.1388	0.5241	-0.2263
Зейделя	0.138808	0.524116	-0.226305

Проверка подстановкой в исходную систему (для решения методом Гаусса):

$$0.9 \cdot 0.1388 + 2.7 \cdot 0.5241 - 3.8 \cdot (-0.2263) = 0.1249 + 1.4151 + 0.8599 = 2.3999 \approx 2.4$$

$$2.5 \cdot 0.1388 + 5.8 \cdot 0.5241 - 0.5 \cdot (-0.2263) = 0.3470 + 3.0398 + 0.1132 = 3.5000 \approx 3.5$$

$$4.5 \cdot 0.1388 - 2.1 \cdot 0.5241 + 3.2 \cdot (-0.2263) = 0.6246 - 1.1006 - 0.7242 = -1.2002 \approx -1$$

.2

Система выполняется с точностью до ошибок округления.

Анализ точности:

- 1) Метод Гаусса даёт решение за конечное число шагов; погрешность обусловлена только округлениями.
- 2) Метод Зейделя даёт приближённое решение; точность определяется заданным $\varepsilon = 0.0001$. Расхождение с методом Гаусса объясняется тем, что итерации остановлены при достижении заданной точности.

Вывод: оба метода дают согласованные результаты. Метод Гаусса предпочтителен для небольших систем ($n \lesssim 103$), метод Зейделя — для больших разреженных систем, где важна экономия памяти и времени.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была решена система линейных алгебраических уравнений **варианта №26** двумя методами: прямым методом Гаусса с выбором главного элемента и нормированием и итерационным методом Зейделя.

Метод Гаусса дал решение $x_1 \approx 0.1388$, $x_2 \approx 0.5241$, $x_3 \approx -0.2263$. Метод Зейделя с точностью $\varepsilon = 0.0001$ сошёлся к тем же значениям за 12 итераций. Оба решения проверены подстановкой в исходную систему — невязка не превышает заданной точности.

Для обеспечения сходимости метода Зейделя исходная система была преобразована линейной комбинацией уравнений к виду с диагональным преобладанием. Проверка нормы матрицы C показала, что $\|C\|_{\infty} = 0.9474 < 1$, то есть условие сходимости выполнено.

В ходе работы были закреплены теоретические знания о прямых и итерационных методах решения СЛАУ, получены практические навыки их реализации на языке C++ и анализа точности вычислений.

Приложение

Листинг программы

```
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

const int n = 3;
const int m = n + 1;
const double EPS = 0.0001;

// Пересчёт индексов (i, j) в одномерный массив
int idx(int i, int j) {
    return i * m + j;
}

void printMatrix(const double a[]) {
    for (int i = 0; i < n; i++) {
        cout << "| ";
        for (int j = 0; j < m; j++)
            cout << setw(10) << setprecision(4) << fixed << a[idx(i, j)] << " ";
        cout << "|" << endl;
    }
    cout << endl;
}

// Метод Гаусса с выбором главного элемента и нормированием
// Метод Гаусса с выбором главного элемента и нормированием
```

```

void gaussJordan(double a[], double x[]) {
    cout << "=== МЕТОД ГАУССА С ВЫБОРОМ ГЛАВНОГО
ЭЛЕМЕНТА И НОРМИРОВАНИЕМ ===\n\n";
    cout << "Исходная расширенная матрица:\n";
    printMatrix(a);

    for (int k = 0; k < n; k++) {
        cout << "--- Шаг " << k + 1 << " ---\n";

        // Поиск главного элемента
        cout << "Ищем главный элемент в столбце " << k + 1 << ".\n";
        int maxRow = k;
        for (int i = k + 1; i < n; i++)
            if (fabs(a[idx(i, k)]) > fabs(a[idx(maxRow, k)]))
                maxRow = i;

        cout << "Максимальный по модулю: a[" << maxRow + 1 << "][" << k
+ 1
        << "] = " << a[idx(maxRow, k)] << "\n";

        // Перестановка
        if (maxRow != k) {
            cout << "Переставляем строки " << k + 1 << " и " << maxRow + 1
<< ":\n";
            for (int j = 0; j < m; j++)
                swap(a[idx(k, j)], a[idx(maxRow, j)]);
            printMatrix(a);
        } else {

```

```

    cout << "Перестановка не требуется.\n\n";
}

```

```

// Нормирование

```

```

double pivot = a[idx(k, k)];
cout << "Нормируем строку " << k + 1
    << ": делим все элементы на " << pivot
    << " (a[" << k + 1 << "][" << k + 1 << "])\n";
for (int j = 0; j < m; j++)
    a[idx(k, j)] /= pivot;
printMatrix(a);

```

```

// Обнуление

```

```

cout << "Обнуляем столбец " << k + 1 << " в остальных строках.\n";
for (int i = 0; i < n; i++) {
    if (i == k) continue;
    double factor = a[idx(i, k)];
    cout << "Для строки " << i + 1 << ": множитель = " << factor
        << ", вычитаем " << factor << " * строку " << k + 1 << "\n";
    for (int j = 0; j < m; j++)
        a[idx(i, j)] -= factor * a[idx(k, j)];
}
printMatrix(a);
}

```

```

for (int i = 0; i < n; i++)
    x[i] = a[idx(i, n)];
}

```

```

// Метод Зейделя
void seidel(const double a[], double x[], double eps) {
    cout << "=== МЕТОД ЗЕЙДЕЛЯ ===\n\n";
    cout << "Система, приведённая к виду с диагональным
преобладанием:\n";
    printMatrix(a);

    // --- Проверка условия сходимости ---
    cout << "Проверка условия сходимости  $\|C\|_{\infty} < 1$ :\n";
    double norm = 0;
    for (int i = 0; i < n; i++) {
        double sum = 0;
        cout << " Строка " << i + 1 << ": ";
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            double c = fabs(a[idx(i, j)] / a[idx(i, i)]);
            sum += c;
            cout << "|a" << i + 1 << j + 1 << "/a" << i + 1 << i + 1 << "|";
            if (j < n - 1) cout << " + ";
        }
        cout << " = " << sum << "\n";
        if (sum > norm) norm = sum;
    }
    cout << "Норма  $\|C\|_{\infty}$  = " << norm;
    if (norm < 1)
        cout << " < 1 — условие сходимости выполнено.\n\n";
    else

```

```
cout << " >= 1 — условие сходимости НЕ выполнено!\n\n";
```

```
// --- Приведение к виду  $x = Cx + d$  ---
```

```
cout << "Приведение к виду  $x = Cx + d$ :\n";
```

```
for (int i = 0; i < n; i++) {
```

```
    cout << "  x" << i + 1 << " = ";
```

```
    bool first = true;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i == j) continue;
```

```
        double c = -a[idx(i, j)] / a[idx(i, i)];
```

```
        if (!first) cout << (c >= 0 ? " + " : " - ");
```

```
        else if (c < 0) cout << "-";
```

```
        cout << fabs(c) << "*x" << j + 1;
```

```
        first = false;
```

```
    }
```

```
    double d = a[idx(i, n)] / a[idx(i, i)];
```

```
    cout << (d >= 0 ? " + " : " - ") << fabs(d) << "\n";
```

```
}
```

```
cout << "\n";
```

```
// --- Начальное приближение ---
```

```
for (int i = 0; i < n; i++) x[i] = 0;
```

```
cout << "Начальное приближение:  $x = (0, 0, 0)$ \n\n";
```

```
cout << "Итерации:\n";
```

```
cout << setw(4) << "k"
```

```
    << setw(14) << "x1"
```

```
    << setw(14) << "x2"
```

```

    << setw(14) << "x3"

    << setw(16) << " погрешность" << endl;
cout << string(62, '-') << endl;

int iter = 0;
double error;
do {
    double x_old[n];
    for (int i = 0; i < n; i++) x_old[i] = x[i];

    cout << "\nИтерация " << iter + 1 << ":\n";

    for (int i = 0; i < n; i++) {
        double sum = a[idx(i, n)] / a[idx(i, i)];
        cout << "  x" << i + 1 << " = " << a[idx(i, n)] << "/"
            << a[idx(i, i)];

        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            double c = -a[idx(i, j)] / a[idx(i, i)];
            double val = (j < i) ? x[j] : x_old[j]; // новые или старые
            cout << (c >= 0 ? " + " : " - ")
                << fabs(c) << "*" << val;
            sum += c * val;
        }
        x[i] = sum;
        cout << " = " << x[i] << "\n";
    }
}

```

```

iter++;
error = 0;
for (int i = 0; i < n; i++) error += fabs(x[i] - x_old[i]);

cout << " Погрешность: ";
for (int i = 0; i < n; i++) {
    cout << "|x" << i + 1 << " - x" << i + 1 << "_old|";
    if (i < n - 1) cout << " + ";
}
cout << " = " << error << "\n";

cout << setw(4) << iter;
for (int i = 0; i < n; i++)
    cout << setw(14) << setprecision(6) << fixed << x[i];
cout << setw(16) << setprecision(6) << fixed << error << endl;

if (error <= eps) {
    cout << "\nПогрешность " << error << " <= " << eps
        << " — итерации завершены.\n";
    break;
}
} while (true);
}

int main() {
    double a[n * m];
    cout << "Введите матрицу " << n << "x" << m << ":" << endl;
    for (int i = 0; i < n; i++)

```

```

        for (int j = 0; j < m; j++)
            cin >> a[idx(i, j)];

double a1[n * m], a2[n * m];
for (int i = 0; i < n * m; i++) a1[i] = a2[i] = a[i];

double x1[n], x2[n];
gaussJordan(a1, x1);

cout << "Решение (Гаусс): ";
for (int i = 0; i < n; i++) cout << x1[i] << " ";
cout << endl;

// Преобразование системы для сходимости Зейделя
for (int j = 0; j < m; j++) {
    a2[idx(0, j)] = a[idx(1, j)] + a[idx(2, j)];
    a2[idx(1, j)] = a[idx(0, j)] + a[idx(1, j)];
    a2[idx(2, j)] = a[idx(0, j)];
}

seidel(a2, x2, EPS);

cout << "Решение (Зейдель): ";
for (int i = 0; i < n; i++) cout << x2[i] << " ";
cout << endl;

return 0;
}

```

ПРИЛОЖЕНИЕ А

Полный код лабораторной работы доступен по ссылке:

https://github.com/ZentoTAG/LABS_DM/blob/main/lab_1/main.py